PROJECT REVIEW FIELD GUIDE

FreshSense AI

A transparent prototype for food freshness assessment using image, optional sensor, and expiry-date evidence.

How to introduce the project, demonstrate its current workflow, explain its technical design, and discuss its evidence responsibly.

Reviewer walkthrough · Prepared for a live software project review

01 / Start with the problem and contribution

Open with the user problem, then frame what this implementation contributes today.

> *“FreshSense brings a food photo, optional environmental readings, and package-date information into one reviewable assessment. Its Freshness Passport is designed to show what evidence contributed, what was missing, and how the recommendation was produced.”*

The problem

People often make freshness decisions from incomplete and scattered signals. A visual estimate alone cannot communicate whether other useful evidence was available or how signals might disagree.

The project idea

Present an image estimate alongside optional temperature, humidity, VOC, and expiry evidence in a traceable workflow, rather than hiding the reasoning behind one unexplained label.

What is built

A React progressive web app, FastAPI backend, PostgreSQL persistence, JWT authentication, image upload/capture flow, scan history, OCR date parsing, sensor API, and deterministic assessment workflow.

What is being investigated

Whether adding expiry and sensor evidence can improve held-out assessments over image-only predictions. This comparison needs paired observations and has not yet been established.

Suggested framing: Treat the Freshness Passport and evidence-handling workflow as the contribution hypothesis. Do not claim scientific novelty or improved accuracy without a related-work review and a suitable independent evaluation.

One-sentence scope

FreshSense is an academic prototype and decision-support workflow. It is not a certified food-safety tool, and an image cannot establish microbial safety, smell, or internal spoilage.

---

02 / Run a focused live demonstration

Aim for a 4–6 minute walkthrough. Show one complete path and narrate the evidence boundary.

> **Before the review**
> Prepare Node.js/npm, Python dependencies, frontend dependencies, a reachable PostgreSQL database, backend environment settings, and database migrations. Start both services with the project launcher. Use a small, non-sensitive sample food image.

```
Windows: start-dev.cmd
macOS/Linux: ./start-dev.sh
Any platform: node start-dev.mjs
```

Frontend opens at **http://127.0.0.1:5173**; API docs are at **http://127.0.0.1:8000/docs**.

1 Introduce the landing page

Point out the image-first flow and the stated role of sensors and expiry information. Note that analysis runs on the server and requires a connection.

2 Sign in or create a review account

Authentication uses the backend API. If the review environment is pre-seeded, use the prepared account. Do not display secrets or real user data.

3 Create one scan

Choose a food photo, optionally enter food type and available temperature, humidity, or VOC readings, then submit. Explain that optional fields are only meaningful when measured and documented; don’t invent readings for the demo.

4 Inspect the result and Passport

Show the returned freshness label, confidence, recommendation, warnings, and available modalities. Identify which items are model estimates and which are rule-based.

5 Close on persistence and transparency

Open history, revisit the scan, and optionally show the API schema in **/docs**. Explain the model status and limitations before taking questions.

> **Demo fallback:** If live upload or the database is unavailable, show the landing page and API documentation, then walk through the prepared screenshots or a recorded run. Clearly label this as a fallback; do not imply that a live scan succeeded.

03 / Explain how the system fits together

Describe the end-to-end data path, then show where each responsibility lives.

> **Browser / PWA** → **FastAPI routes** → **services** (scan, OCR, sensor, assessment) → **vision / parsing / deterministic decision** → **PostgreSQL records** → result and history in the frontend

Layer	Role to explain	Review cue
React + Vite PWA	Landing, account screens, dashboard, food camera/upload, optional expiry-label close-up, results, and history.	Show responsive scan form and online-only inference notice.
FastAPI API	Authentication, image validation, scan and prediction routes, OCR, sensors, assessments, health.	Open `/docs`; point to multipart scan and health endpoints.
Assessment services	Compose available evidence and apply deterministic rules for the current decision flow.	Explain what evidence was present and what was missing.
PostgreSQL + SQLAlchemy	Persist users, scans, image predictions, OCR and sensor records, and assessments.	Migration ownership is Alembic; keep application and test databases separate.
Model interface	Uses a local candidate artifact if available; otherwise falls back to the explicitly identified prototype path.	Verify the active configuration before the review; no weights are committed.

Important distinction: inputs, estimates, and rules

- **Image estimate:** prototype heuristic unless a reviewed local model artifact is installed and configured.
- **OCR:** RapidOCR reads the food photo or an optional close-up of the expiry label. It is best-effort; the supplied angled, watermarked sample did not produce a confirmed date. Raw recognized text is shown, and dates should be checked against the package.
- **Sensor values:** API and provider abstractions exist; calibrated physical sensor hardware is future work.
- **Fusion:** deterministic rules today; learned multimodal fusion and shelf-life regression are not implemented.

04 / Present evidence and limits clearly

A strong review distinguishes functioning software from validated predictive performance.

Evidence	What it supports	What it does not establish
End-to-end application workflow and API	Integration of scan submission, assessment, persistence, and retrieval.	Accuracy or real-world usefulness of the freshness output.
Candidate image experiment	Training path exercised on a convenience sample; best candidate reported 52.6% accuracy and 53.1% macro recall on 445 held-out AgriFreshNET images.	Independent benchmark performance. The validation split has been reused during model comparison; results are exploratory. Candidate is inactive.
Public image datasets	Image-only experimentation across produce freshness labels, with source attribution recorded.	Paired sensor/expiry benefit, packaged-food generalization, or food safety.
Fresh and Rotten supplement	3,200 raw images, 1,600 fresh and 1,600 spoiled, added locally as training-only data.	Moderately-fresh classification or leakage-safe validation; source lacks that class and reliable session IDs.

> **Say this plainly:** “The result is an informational estimate, not a safety determination. The strongest model candidate remains inactive; unless a local reviewed artifact is configured, the app uses the labeled prototype fallback.”

Evaluation needed for the next research claim

- Collect independently reviewed freshness labels and paired readings from the same item and capture time.
- Split by item, batch, or capture session before model tuning; keep groups disjoint.
- Compare image-only with the current Passport rules on an untouched grouped validation set.
- Report per-class metrics, especially spoiled recall, confusion matrices, group/sample counts, missing-modality rates, and uncertainty.
- Review related work before presenting novelty as anything more than a hypothesis.

The local dataset images, manifest, and model artifacts are ignored by Git and are not bundled with the repository. Preserve dataset attribution and CC BY 4.0 terms when using or redistributing derived material.

05 / Close with a review-ready summary

> *“The current milestone demonstrates a full-stack freshness-assessment workflow with transparent evidence handling. It establishes an engineering foundation and a reproducible image-training path. The learned model, calibrated sensors, paired-data comparison, and shelf-life prediction remain future validation work.”*

Likely reviewer questions

Question	Concise, evidence-aligned response
Is the model trained?	A local candidate was trained, but it remains inactive. The app uses a prototype fallback unless a reviewed artifact is installed and configured.
How accurate is it?	The best candidate reached 52.6% accuracy and 53.1% macro recall on a small reused validation split. Those scores are exploratory, not an independent generalization result.
Does sensor fusion improve results?	That has not been shown. Public image sources lack paired sensor or expiry observations; collecting grouped paired data is the next evaluation step.
Can it tell food is safe?	No. It is an informational assessment and cannot detect microbial hazards or establish safety.
What is the novelty?	The proposed contribution is evidence provenance and conflict-aware presentation in the Freshness Passport. Novelty needs a literature review; effectiveness needs a paired study.
What comes next?	Collect paired, reviewed observations; build a new group-disjoint benchmark; calibrate sensors; compare image-only and Passport outputs; then consider learned fusion and edge deployment.

Final setup checklist

- Confirm `backend/.env` contains valid PostgreSQL URLs and a strong JWT secret; never put credentials in slides or screenshots.
- Apply migrations, confirm health endpoint, and create a review user or seed data.
- Start both services with `start-dev.cmd`, `./start-dev.sh`, or `node start-dev.mjs`.
- Test one small image and verify the result/history route before the review starts.
- Have a prepared fallback demo and keep the prototype and safety disclaimers visible.

Project references: README.md · ARCHITECTURE.md · API.md · NOVELTY.md · data/DATASETS.md · backend/AI_MODEL.md